Wolf and Rosenstein Across Two Reference Catalogues

A lab report on largest-Lyapunov-exponent estimation: how the data were built, what went wrong along the way, how accurate the estimators are, and how they respond to noise

Quantitative Analysis Research Core (QUARC) — QUARC Nonlinear

2026-10-12

Table of contents

1 Summary 2
2 Purpose 2
3 Reference systems 3
3.1 Sprott (2003), Appendix A 3
3.2 dysts (Gilpin, 2021) 3
3.3 What counts as the reference exponent 3
4 Data construction 4
4.1 Sprott series (generated in MATLAB) 4
4.2 dysts series (generated in Python) 4
4.3 Protocol side by side 5
5 Estimators and analysis protocol 5
5.1 Phase-space reconstruction 5
5.2 Wolf's method 5
5.3 Rosenstein's method 5
5.4 Summary statistics 6
6 Trial and error 6
7 Results 7
7.1 Accuracy across both catalogues 7
7.2 Wolf versus Rosenstein, head to head 11
7.3 Rosenstein's scaling regions 11
7.4 Wolf's renormalisation interval 12
7.5 The effect of the reference choice 13
7.6 One method for both catalogues 14
7.7 Sensitivity to observational noise 16
7.8 The RQA family under the same noise 18
8 Discussion 20
9 Limitations and open issues 20
10 Data assets and how they can be reused 21
11 Reproducing this report 22
12 Appendix A: per-system results, Sprott catalogue 23
13 Appendix B: per-system results, dysts catalogue 24
Bibliography 28

1 Summary

This report collects the work done in the QUARC Nonlinear repository to establish how well the two largest-Lyapunov-exponent estimators in the library, Wolf's method (Wolf et al., 1985) and Rosenstein's method (Rosenstein et al., 1993), recover known exponents across a wide range of dynamical systems, and how that recovery degrades under observational noise. It is written as a lab record. The trial and error is part of the result, not a preamble to it.

Two reference catalogues, one protocol. The estimators were run on 58 usable systems from Appendix A of Sprott (2003) (maps and flows, 1000 realizations each, 4000 samples) and 126 continuous-time systems from the dysts database of Gilpin (2021) (100 realizations each, 16000 samples). Every system was analysed under the same embedding and fitting protocol, with no per-system tuning.

Headline accuracy. Expressed as the ensemble-median estimate divided by the reference exponent, then summarised across systems:

- On Sprott's catalogue, Wolf reads 0.92 (interquartile range 0.76–1.09) and Rosenstein 0.84 (0.75–0.92).
- On dysts, Wolf reads 0.93 (0.56–1.23) and Rosenstein 0.84 (0.58–1.46).
- Rosenstein reads low by about 16% on *both* catalogues. Its automatically fitted scaling regions are concave in 98% of Sprott systems and 90% of dysts systems, which is what fitting across the knee into saturation looks like. That is a plausible mechanism for the bias.
- Wolf is closer to the reference than Rosenstein on 82 of 123 dysts systems and 34 of 58 Sprott systems. But its worst misses are far larger: about 89,944-fold on DoublePendulum, against Rosenstein's worst of about 362-fold on ForcedVanDerPol.

Noise. Both estimators hold near their clean values up to about 2% additive noise (relative to the series' standard deviation). Above that they fail in *opposite* directions. At 10% noise, with the clean embedding held fixed, Wolf overshoots to 1.65 and Rosenstein falls to 0.67.

Re-estimating the embedding on the noisy series — what a practitioner would actually do — pulls both Lyapunov estimates *down*. That deepens Rosenstein's low bias (0.50) and partly offsets Wolf's overshoot (1.40): two errors cancelling, not a fix. The embedding change runs through an inflated FNN dimension, not the AMI delay.

The most consequential correction was to the reference values themselves. The dysts reports were first compared against dysts' trajectory-divergence estimate of the exponent, which is itself an estimate, rather than against the tangent-space value that the estimators are estimating. Correcting that moved the median Wolf ratio from 0.79 to 0.93.

One method for both references. The two catalogues' published exponents come from different procedures, and on the 13 systems both hold they disagree by a median of 41%. Recomputing every reference under one method (Section 7.6) collapses that to within 3% wherever the two catalogues use the same control parameters, and shows the published dysts spectra to be 19% high at the median. Re-dividing the committed estimates by the recomputed references leaves the Sprott results unchanged and moves the dysts ones: Rosenstein's dysts median goes from 0.86 to 0.97, so most of its apparent low bias on that catalogue was the reference, while Wolf's goes from 0.98 to 1.08. The same comparison exposed one transcription error in the Sprott catalogue (ACT) that no internal check could have caught.

2 Purpose

The largest Lyapunov exponent, λ_1, measures the average exponential rate at which nearby trajectories of a dynamical system separate (Eckmann & Ruelle, 1985). Positive λ_1 is the operational signature of chaos, and estimating it from a single measured time series is one of the most common nonlinear analyses in movement science and elsewhere. Wolf's and Rosenstein's algorithms are the two most widely used estimators, and both ship in this library.

Published validations of these estimators typically use a handful of textbook systems — Lorenz, Rössler, Hénon, the logistic map. That leaves three questions open:

1. **How accurate are they across a broad, diverse catalogue**, where the reference exponent is known independently of any time-series estimator?
2. **Where and why do they fail**, and are the failures sampling noise or systematic bias?

3. **How robust are they to observational noise**, and how much of the damage comes from the estimator versus from the embedding step that precedes it?

The work reported here was built as a *characterization* instrument, not a pass/fail test suite. Nothing in it asserts a tolerance. It records how each estimator behaves on each system, with the spread across an ensemble of initial conditions, so that changes to the library can be judged by diffing two reports.

3 Reference systems

3.1 Sprott (2003), Appendix A

Appendix A of Sprott (2003) catalogues 62 low-dimensional chaotic systems, each with equations, standard parameters, an initial condition, and a published largest Lyapunov exponent in nats per unit time (or per iteration, for maps). Four were unusable and say why:

- **Spence map:** the exponent diverges.
- **Tent map and Kaplan–Yorke map:** these are exact binary shifts, so double precision exhausts the mantissa in about 52 iterations regardless of the initial condition.
- **Gaussian white chaotic map:** its transcription could not be verified.

That leaves 58 systems, spanning noninvertible, dissipative and conservative maps and autonomous, driven and conservative flows. Building the catalogue surfaced three transcription errors, which were fixed:

- Chua's bracketed terms are added, not subtracted. Subtracting them drives the system to a fixed point.
- Burke–Shaw needs $+Uxy$ in $\dot{z}$. The other sign diverges at every step size.
- The Gauss map cannot start at Sprott's $x_0 = 0.1$, because $1/0.1$ rounds to exactly 10 in binary and the orbit dies immediately.

3.2 dysts (Gilpin, 2021)

The dysts database catalogues 135 continuous-time chaotic systems in a Python package; version 0.96 was used here. For each system it publishes a Lyapunov spectrum computed by integrating the tangent space along a trajectory with Gram–Schmidt/QR re-orthonormalization (Benettin et al., 1980; Christiansen & Rugh, 1997), along with a correlation dimension and other invariants.

Six of the 135 are delay-differential equations (IkedaDelay, MackeyGlass, PiecewiseCircuit, ScrollDelay, SprottDelay, VossDelay). They need a history function rather than an initial condition, and were excluded. Of the remaining 129:

- **ArnoldBeltramiChildress** has no bounded state component. Delay embedding requires a bounded, recurrent observable, so it was excluded.
- **SprottMore** and **StickSlipOscillator** lost every realization to degeneracy during integration.

That leaves 126 systems: 108 autonomous flows, 9 driven (nonautonomous) flows, and 9 conservative (Hamiltonian) flows.

Trajectories were generated in Python by dysts itself and read from disk in MATLAB, rather than porting the vector fields by hand. Hand-porting 126 systems would put the data and its reference values on different provenances, and a mistyped coefficient still produces a perfectly plausible chaotic series that nothing downstream would flag.

3.3 What counts as the reference exponent

For Sprott, the reference is the published λ_1.

For dysts, the reference is the largest element of the tangent-space spectrum, `lyapunov_spectrum_estimated`. dysts also publishes `maximum_lyapunov_estimated`, which it obtains by measuring the divergence of nearby trajectories. That is an estimate of the same quantity by the *same kind of method* the library's estimators use, so comparing against it would score one estimator against another.

The two numbers are far apart. Across the 126 systems they agree within 10% on only 31, differ by more than 50% on 43, and on ForcedVanDerPol the tangent-space value is 75.6 times the divergence value. Torus has a largest tangent-space exponent of exactly zero — it is quasi-periodic rather than chaotic — so no ratio can be formed for it.

This choice was not made correctly the first time; see Section 6.

Both reference sets carry their own uncertainty. The dysts spectra are numerical, and Sprott's values are published to limited precision. The ratios reported here therefore bound the disagreement between estimator and reference together, not estimator error alone.

They also come from two different procedures, which is a confound of its own: a ratio against a Sprott value and a ratio against a dysts value are not measuring the same thing. Section 7.6 recomputes every reference in both catalogues under one method and re-divides the committed estimates by the result. The tables in the rest of this report keep the published references, so that they remain comparable with the earlier record; the recomputed ones are the better reference going forward.

4 Data construction

Both catalogues follow one design. Each system gets an ensemble of initial conditions. Realization 1 always starts from the published initial condition, verbatim, and the rest are drawn *on the attractor*. Every series is sampled at about 40 points per dominant period. The ensemble is what turns a deterministic system into a distribution, so that systematic bias can be told apart from sampling variability.

4.1 Sprott series (generated in MATLAB)

- **Flows** are integrated with a fixed-step fourth-order Runge–Kutta scheme at the catalogue's step size. A 20,000-step transient is discarded. Output is decimated so the dominant spectral period lands near 40 samples.
- **Maps** are iterated directly after a 1,000-iteration transient.
- **Ensemble initial conditions:** run 5,000 steps from the published x_0 to land on the attractor; measure the attractor's per-component standard deviation; perturb by 1% of it (`Spread = 0.01`); relax the perturbed point for 200 steps.
- **Decimation is fixed per system**, computed once from the published x_0 and applied to every realization.
- **Size:** 1000 realizations of 4000 samples per system. Base seed 20260810.

The full-catalogue run took 8.95 h on 32 workers. 53 of 58 systems delivered all 1,000 realizations. The cusp map lost 1 and the simplest quadratic flow lost 5. The damped pendulum (460 usable), driven pendulum (387) and labyrinth (440) degenerate from Sprott's own initial condition, because their observable carries an unbounded drift. Where the loss is large, the survivors are a sample conditioned on not degenerating.

4.2 dysts series (generated in Python)

- **Integrator:** dysts' default, SciPy's `solve_ivp` (Virtanen et al., 2020) with the implicit Radau IIA method (Hairer & Wanner, 1996), chosen by dysts because several catalogue systems are stiff. Trajectories are resampled to 40 points per dominant period.
- **Ensemble initial conditions** mirror the Sprott design. Integrate 50 dominant periods from the published initial condition; take the per-component standard deviation of that anchor trajectory as the attractor's extent; perturb the final point by 1% of it; relax for 5 periods.
- **Observable:** the first state component not flagged unbounded.
- **Size:** 100 realizations of 16000 samples (400 dominant periods) per system, stored as float64. The corpus is about 1.6 GB.
- **N is a ceiling.** Truncating a stored series is free and lengthening one means regenerating everything, so series-length experiments are column slices.
- **float64, not float32.** The correlation-dimension estimator works on inter-point distances at the smallest radii, and disk is cheaper than a question about precision.
- **A 900 s wall-clock cap per solve.** A solve that has not returned by then is recorded as a degenerate realization (see Section 6). HyperLu has 2 and Sakarya has 3 degenerate realizations of 100; every other system is complete.

The export took on the order of 17 h on 30 workers; the characterization battery over it took 10.3 h. The `.bin` files are gitignored and shared via Dropbox (`quarc-data/dysts`). The manifest, which carries reference values and protocol, is committed.

4.3 Protocol side by side

	Sprott (2003)	dysts (Gilpin, 2021)
Systems analysed	58 (maps and flows)	126 (flows only)
Generated in	MATLAB	Python (dysts 0.96)
Integrator	RK4, fixed step	Radau IIA, adaptive (SciPy)
Sampling	~40 samples per dominant period	40 points per dominant period
Realizations	1000	100
Samples per series	4000	16000
Realization 1	Published x_0	Published initial condition
Other realizations	On-attractor, 1% of attractor SD	On-attractor, 1% of attractor SD
Reference λ_1	Published value	Max of tangent-space spectrum

5 Estimators and analysis protocol

5.1 Phase-space reconstruction

A scalar series is embedded by delay coordinates (Takens, 1981). The embedding is a *protocol*, stated per kind of system, not a per-system tuning:

- **Flows:** the delay is the first minimum of the average mutual information (Fraser & Swinney, 1986). The dimension is chosen by false nearest neighbours (Kennel et al., 1992), searched at that delay. If AMI yields no usable minimum, the delay falls back to 1. If FNN returns less than the true state dimension, the dimension falls back to state dimension + 1.
- **Maps:** delay 1 and dimension = state dimension + 1. A map's iterates are its natural coordinates, so AMI's first minimum on a map is not an embedding delay (see Section 6).

AMI and FNN are always reported as metrics in their own right, next to the delay and dimension actually used.

5.2 Wolf's method

Wolf's algorithm (Wolf et al., 1985) follows a reference trajectory and its nearest neighbour for a fixed propagation interval (`evolve`, 10 samples here; the library no longer has a default), records the log-stretch, then replaces the neighbour with one near the evolved reference point. The exponent is the average log-stretch per unit time. The library's implementation accumulates $\log_2$ and returns bits; the `lyapunov()` wrapper converts to nats so that both estimators, and all references, share units.

5.3 Rosenstein's method

Rosenstein's algorithm (Rosenstein et al., 1993) pairs each reconstructed point with its nearest neighbour, excluding temporally close candidates within a Theiler window (Theiler, 1986). The default half-width is $\mathrm{round}(0.8\,\tau)$. It then averages the log distance of every pair as a function of time. The exponent is the slope of that mean log-divergence curve over its initial linear region, before saturation.

That slope depends heavily on *where* it is fitted. On one Lorenz series, samples 2–30 give 1.67 and samples 10–150 give 0.74, from identical data. The library therefore selects the region automatically. It locates saturation, scores every candidate window before it (at least 4 and at most 200 points) by the R^2 of a straight-line fit, and takes the *longest* window whose R^2 is within 0.02 of the best. The chosen window is returned so it can be audited.

Because R^2 does not measure linearity (see Section 6), each fit also reports:

- **curvature:** the quadratic coefficient scaled by the window's extent, read as the fraction of the window's rise contributed by the bend;
- **maximum deviation:** the largest residual as a fraction of the rise;
- **runs z-score:** a Wald–Wolfowitz statistic on the residual signs (Wald & Wolfowitz, 1940), which detects systematic departure from a line without assuming it is quadratic.

5.4 Summary statistics

For each system and estimator, the ensemble is summarised by its median and median absolute deviation. Accuracy is the ratio of the ensemble median to the reference. Across systems, the ratio is summarised by its median and interquartile range, the share of systems within a factor of 2 of the reference, and the median $|\log_{10}\text{ratio}|$. The last is symmetric in over- and under-estimation. A negative ratio means the estimator returned the wrong sign for a system whose reference is positive.

Software: MATLAB R2025b (Update 5); Python with dysts 0.96, SciPy 1.18.0 and NumPy 2.5.2; R 4.5.0 with ggplot2 4.0.0 for figures; Quarto 1.9.38 for this report.

6 Trial and error

Most of what makes these numbers trustworthy came from finding things that were wrong. The table below lists every material correction, in roughly chronological order, with the evidence that exposed it. Several produced *plausible* numbers while wrong — the dangerous kind of defect.

#	What was wrong	How it showed up	What changed
1	Wolf returned bits; references are in nats	A correct answer looked like a 44% error (factor ln 2 = 0.693)	The `lyapunov()` wrapper converts both estimators to nats
2	Rosenstein's temporal-neighbour exclusion used a literal distance of 1e5	Not scale invariant: once real distances exceeded 1e5, every point paired with its own time neighbour	Exclusion by index (Theiler window), exposed as a parameter
3	A fixed Rosenstein fitting window	Samples 2–30 gave 1.67 and 10–150 gave 0.74 on identical Lorenz data	Automatic scaling-region selection, returned for audit
4	R^2 used as evidence a fit was straight	A real bend scored R^2 = 0.999926 and straight-plus-noise 0.999933, while their curvatures differed sixteenfold	Curvature, maximum deviation and runs statistics added to every fit
5	Maps embedded with AMI/FNN, like flows	Logistic-map correlation dimension 3.09 against a published 1.0	Maps use delay 1, dimension = state dimension + 1
6	Ensemble initial conditions perturbed around the published x_0	47% of delayed-logistic, 42% of simplest-quadratic and 30% of simplest-cubic realizations escaped the basin	Draw on the attractor, then relax
7	Flow decimation derived per realization	A perturbed x_0 could shift the period estimate and change the sampling rate between realizations	Decimation fixed once per system
8	Suspected transient contamination in heavily decimated flows	Some flows got only ~5 dominant periods of transient	Tested, not assumed: a 10× longer transient moved `corr_dim` ≤ 1.5% and Rosenstein ≤ 3.5%, inside ensemble spread. Unchanged
9	Wolf's `evolve` = 10 applied to maps	0.33× on Arnold's cat, 0.63× on Ikeda; 12–36% error on simple maps	Swept (Section 7.4). Default removed: callers must now choose `evolve`

#	What was wrong	How it showed up	What changed
10	Hand-porting 129 dysts vector fields was the first plan	Data and references would have had different provenances	Generate with dysts itself; read from disk
11	float32 storage	Precision at the smallest radii used by `corr_dim`	float64
12	One perturbed Sakarya solve never returned	110 CPU-hours on a single solve; the export blocked for 44 h	900 s wall-clock cap; wedged solves recorded as degenerate
13	IC seeds used Python's salted `hash()`	The same `--seed` drew different initial conditions in every process	`crc32` of the system name
14	The manifest was written only on completion	The interrupted export left 125 binaries that no longer described themselves	Manifest rebuilt from the binaries; the exporter now appends each entry as it lands
15	dysts compared against its divergence estimate, not the tangent-space value	The code's own documentation said tangent-space; the data did not match	Reference switched (Section 3.3, Section 7.5)
16	Noise figure drew one line per system	At 126 systems a few outliers (ratios of 100 and −150) set the axis	Interquartile bands across systems
17	Noise-sweep validation against an older per-realization file	Only about 75% of perturbed realizations matched	Validated against live code instead: 700 of 700 values exact
18	The noise result was first written up as "re-embedding is the worse arm in all three metrics"	At 10% noise Wolf's re-embedded median is closer to the reference than its fixed one, and the share of systems within 2× is similar between arms up to 5%	Stated per estimator and per criterion here (Section 7.7); the repository README was corrected to match (commit `f9419b1`)

Row 17 deserves a note. The older per-realization table (`quarc-data/derived/characterization_dysts_per_realization.csv`, dated 13 September) matches current results exactly for realization 1 but for only about three quarters of the perturbed realizations. Its provenance has not been established. It should not be used as a reference until it has.

7 Results

7.1 Accuracy across both catalogues

Table 1: Accuracy by catalogue and estimator. Ratio = ensemble-median estimate ÷ reference. IQR and 'within 2×' are across systems.

Catalogue	Estimator	Systems	Median ratio	IQR	Within 2×	Median |log10 ratio|	Wrong sign
Sprott	Wolf	58	0.92	0.76–1.09	76%	0.091	0
Sprott	Rosenstein	58	0.84	0.75–0.92	78%	0.095	0
dysts	Wolf	124	0.93	0.56–1.23	67%	0.159	1
dysts	Rosenstein	125	0.84	0.58–1.46	59%	0.200	1

Both estimators centre slightly below the reference on both catalogues. The dysts catalogue is harder: its interquartile ranges are at least twice as wide as Sprott's, and fewer systems fall within a factor of 2. Part of that is composition.

Sprott's appendix favours minimal low-dimensional forms, while dysts adds stiff, hyperchaotic and higher-dimensional systems.

Table 2: Accuracy by category: median ratio and share of systems within a factor of 2. Systems counts those with a finite ratio.

Catalogue	Category	Systems	Wolf ratio	Wolf within 2×	Rosenstein ratio	Rosenstein within 2×
Sprott	Noninvertible map	9	0.81	78%	0.87	89%
Sprott	Dissipative map	10	0.80	80%	0.82	100%
Sprott	Conservative map	6	0.57	50%	0.97	67%
Sprott	Autonomous flow	20	1.09	90%	0.87	80%
Sprott	Driven flow	8	0.92	88%	0.71	75%
Sprott	Conservative flow	5	1.17	20%	1.33	20%
dysts	Autonomous flow	107	0.98	75%	0.85	67%
dysts	Driven flow	9	0.71	33%	1.86	22%
dysts	Conservative flow	9	0.19	11%	0.14	0%

The category breakdown shows where each estimator struggles. **Conservative flows** are the consistent weak point on both catalogues: their exponents are small, and the noise floor of a finite-length estimate dominates. On dysts, **driven flows** are nearly as hard. **Conservative maps** are hard for Wolf but not for Rosenstein. On Sprott's other maps, Wolf's accuracy depends strongly on its renormalisation interval (Section 7.4).

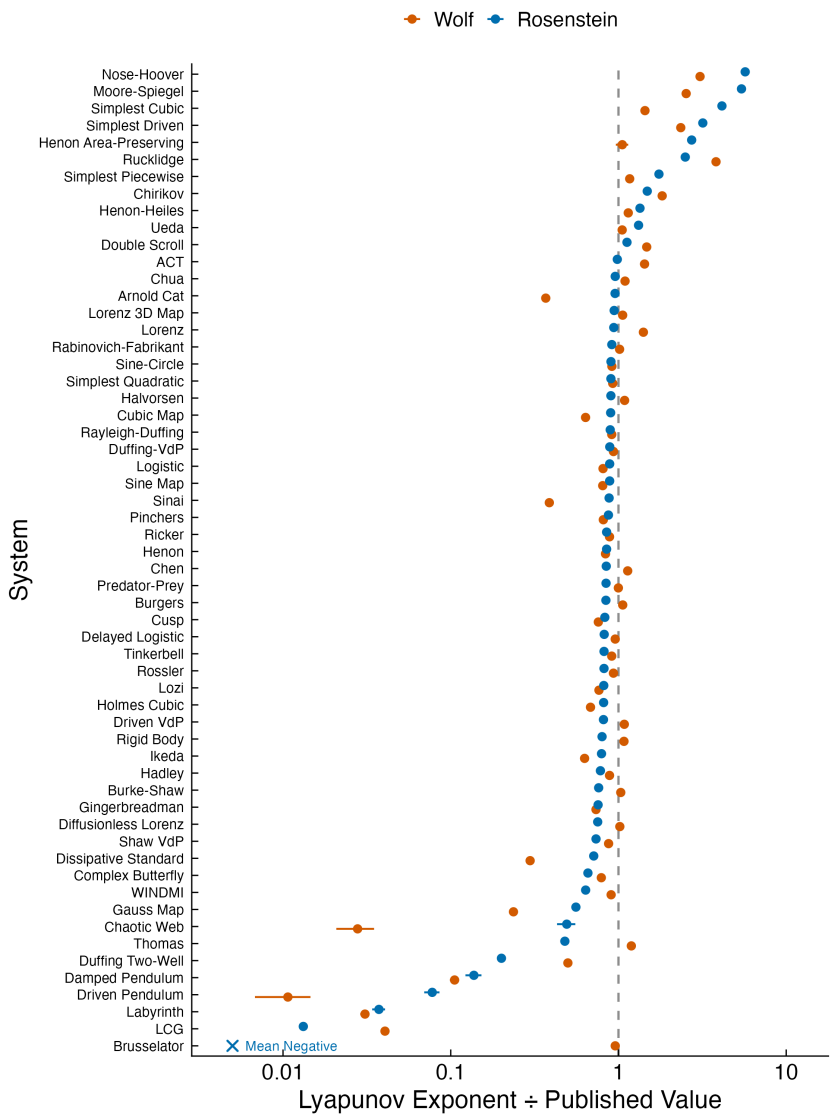


Figure 1: Sprott (2003) catalogue: largest Lyapunov exponent from each estimator divided by the published value, one row per system. Points are ensemble means, whiskers 95% confidence intervals; the log axis spans three orders of magnitude.

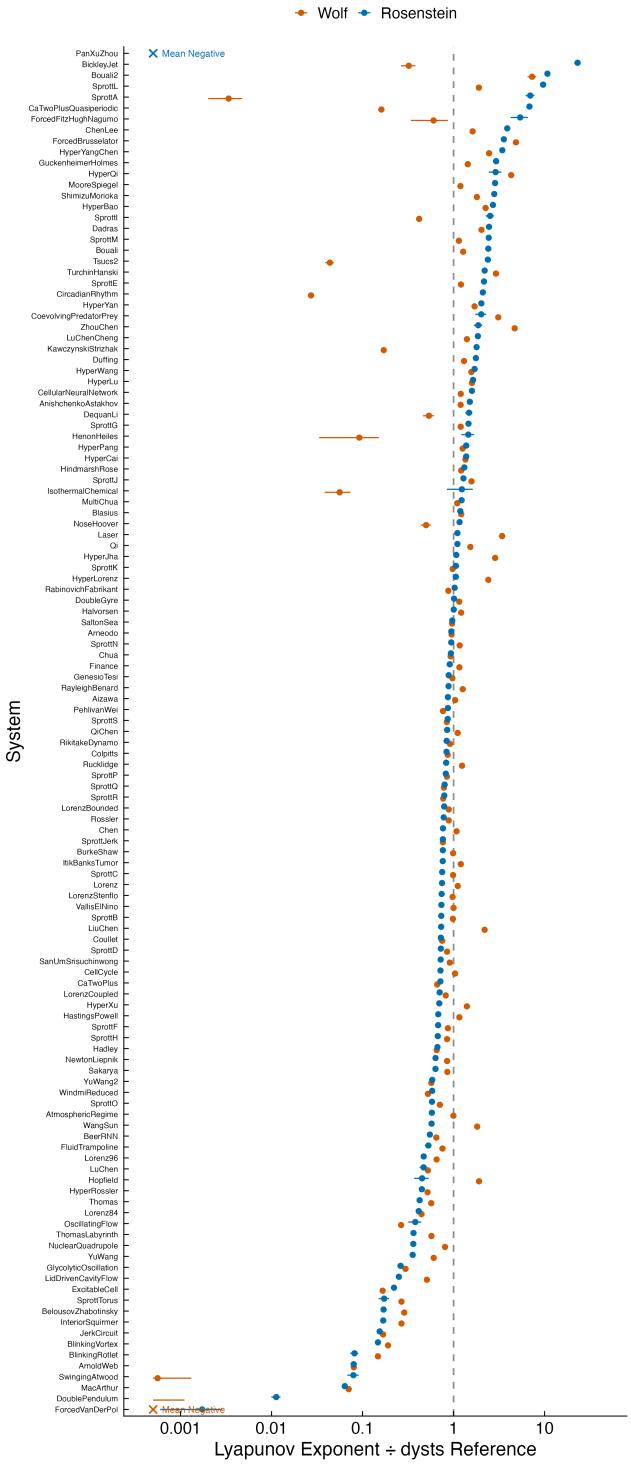


Figure 2: dysts catalogue: largest Lyapunov exponent from each estimator divided by the tangent-space reference, one row per system, ordered by Rosenstein's ratio. Systems whose mean estimate is negative are pinned to the left edge and labelled.

On most systems in Figure 1 the confidence interval is smaller than the marker, which is the point of the ensemble design. The offset from the reference is *systematic*, not sampling noise. In the full Sprott run, the gap between the ensemble median and the reference exceeded ten times the ensemble MAD on 19 of 58 systems for Wolf and 24 of 58 for Rosenstein.

7.2 Wolf versus Rosenstein, head to head

Table 3: Head-to-head comparison on systems where both estimators return a positive ratio. p is a paired Wilcoxon signed-rank test on |log10 ratio|.

Catalogue	Systems	Wolf closer	Median |log10 ratio|, Wolf	Median |log10 ratio|, Rosenstein	p
Sprott	58	34 (59%)	0.091	0.095	0.394
dysts	123	82 (67%)	0.159	0.200	0.00348

Neither estimator dominates. On Sprott's catalogue they are at near parity. On dysts, Wolf is closer on about two systems in three, and its median error is smaller.

The difference is in the tails. Wolf's worst dysts misses are about 89,944-fold on DoublePendulum and 4,708-fold on SwingingAtwood, both far below the reference. Rosenstein's worst is about 362-fold, on ForcedVanDerPol. On Sprott's catalogue the worst misses are about 1,912-fold for Wolf (driven_pendulum) and 82-fold for Rosenstein (chaotic_web).

7.3 Rosenstein's scaling regions

Table 4: Rosenstein fit diagnostics, summarised across systems (medians of per-system ensemble medians). ρ is the Spearman correlation with |log10 ratio|.

Catalogue	Fit length (samples)	At 200-sample cap	R^2	Curvature	Concave	ρ, curvature	ρ, R^2
Sprott	40	12%	0.981	−0.50	98%	−0.16	−0.51
dysts	140	40%	0.980	−0.51	90%	−0.21	−0.34

Rosenstein's fitted regions are bent by about half their own rise, and concave in the large majority of systems on both catalogues. A concave fit across the knee of the divergence curve into saturation underestimates the initial slope, which matches the estimator's catalogue-wide low bias. The regions look excellent by R^2 — median about 0.98 — and that is exactly why R^2 was not trusted as the diagnostic.

On these smooth, low-noise curves, curvature is *not* a better predictor of error than R^2. The two largely coincide, because nearly all of R^2's shortfall *is* curvature. The case for curvature is interpretability, and robustness when noise is present, not better correlation on this set.

On dysts, with series four times longer, the selected windows are much longer and a substantial share hit the 200-sample cap. The cap itself is a protocol choice worth revisiting.

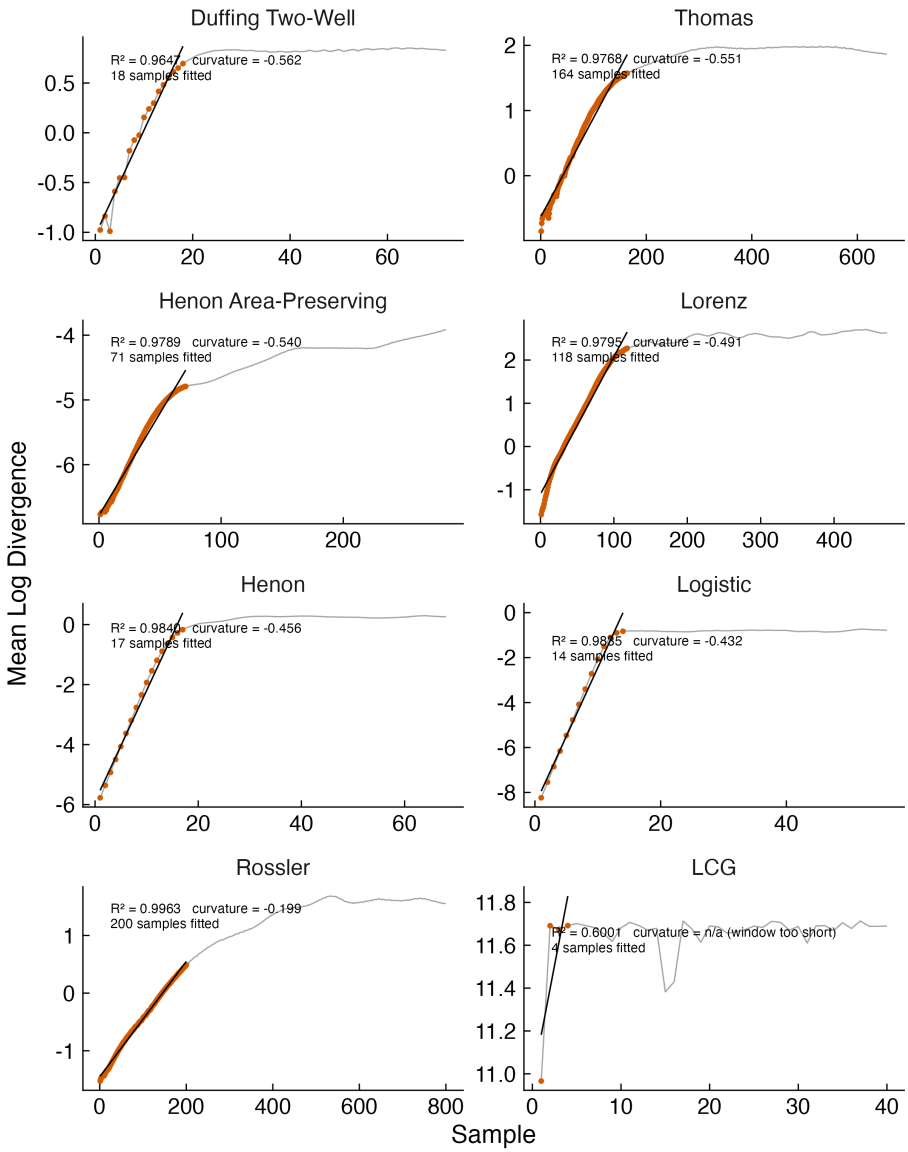


Figure 3: Rosenstein divergence curves for a panel of Sprott systems, with the automatically selected scaling region highlighted, ordered by curvature.

7.4 Wolf's renormalisation interval

Table 5: Wolf's method: median |log10 ratio| across systems, by renormalisation interval (evolve, in samples). Lower is better. 200 realizations per system.

Group	Sys-tems	1	2	3	5	8	10	15	20	30
Noninvertible map	9	0.007	0.011	0.019	0.022	0.106	0.093	0.151	0.270	0.447
Dissipative map	10	0.012	0.013	0.017	0.041	0.079	0.096	0.159	0.256	0.425
Conservative map	6	0.244	0.239	0.239	0.252	0.321	0.357	0.365	0.332	0.360
Flows	8	0.033	0.035	0.053	0.053	0.042	0.036	0.030	0.030	0.024

Wolf's method assumes a neighbouring pair stays in the linear regime over the propagation interval. On a flow sampled at about 40 points per orbit, 10 samples is a quarter turn and the estimate barely depends on it. On a map, 10 samples is 10 iterations: on the logistic map a pair separates by 2^{10} before replacement, far outside any linear regime.

The sweep bears this out:

- Noninvertible and dissipative maps are roughly an order of magnitude more accurate at `evolve` = 1 than at `evolve` = 10.
- Flows are flat across the whole range, which is what separates "maps need a shorter interval" from "the default is wrong everywhere".
- Conservative maps do not follow the pattern. Their small exponents are dominated by the estimation noise floor, and the chaotic web is not estimable this way at any interval.

The library default was deliberately left at 10: changing it would silently alter every exponent the library has produced. The evidence is in `lyapunov()`'s documentation so a caller analysing a map can choose deliberately.

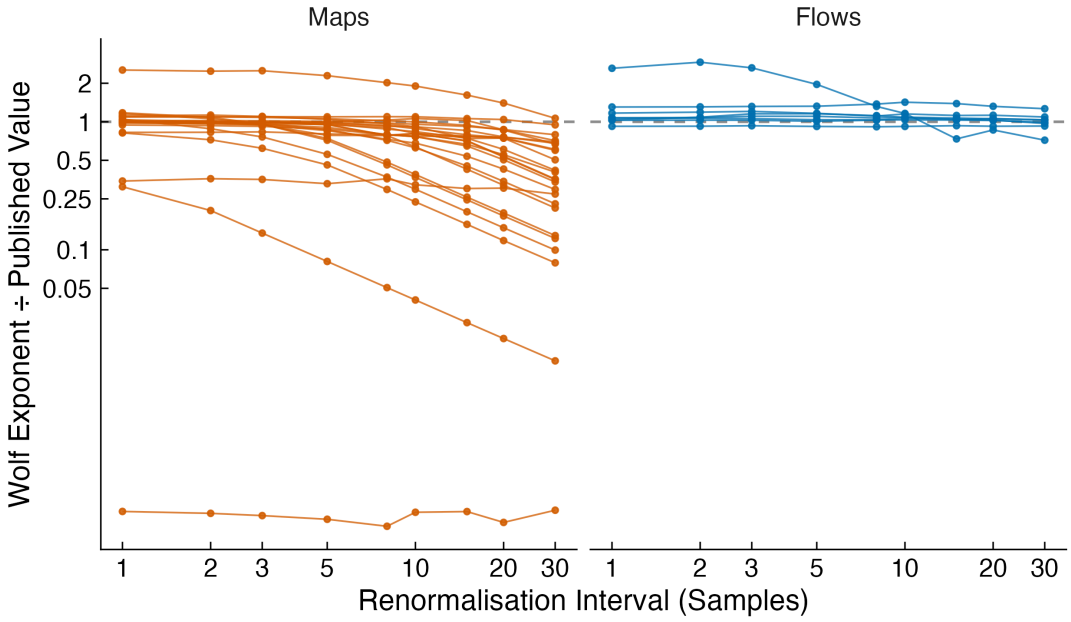


Figure 4: Wolf's estimate against its renormalisation interval, by system group.

7.5 The effect of the reference choice

Table 6: dysts accuracy before and after correcting the reference exponent. Only the reference changed; every estimate is identical.

Estimator	Median ratio, divergence reference	Median ratio, tangent-space reference	Within 2×, before	Within 2×, after
Wolf	0.79	0.93	60%	67%
Rosenstein	0.73	0.84	54%	59%

Re-referencing changed only the `reference` and `medianOverReference` columns of the report. The report was rebuilt from the existing table rather than by re-running the battery. The rebuild was validated first by regenerating the unchanged report from its own table and confirming it was byte-identical. Agreement improved on both estimators, which is what one would expect if the new reference is the right one.

7.6 One method for both catalogues

The switch above corrected *which* dysts number was used. A larger problem remained: the two catalogues' references come from different procedures. Sprott computed his by trajectory separation with periodic renormalisation; dysts integrates the tangent space with QR orthonormalisation. Whether the two agree had never been checked, and the 13 systems that appear in both catalogues make the check direct.

Table 7: The systems both catalogues hold. Before: ratio of the two published exponents (dysts ÷ Sprott). After: the same ratio with both sides recomputed under one method from each catalogue's own initial condition and parameters. Where the parameters match, the disagreement was the procedure; where they differ, what remains is the parameters.

System	Same parameters	Sprott published	dysts published	Ratio, published	Ratio, one method
thomas	no	0.0349	0.759	21.75	14.07
moore spiegel	no	0.112	0.407	3.64	1.75
nose hoover	no	0.0138	0.0476	3.45	0.17
rucklidge	yes	0.0643	0.183	2.84	1.00
hadley	no	0.167	0.312	1.87	1.19
henon heiles	no	0.045	0.0267	0.59	0.02
chua	no	0.327	0.461	1.41	1.34
halvorsen	no	0.79	0.586	0.74	0.86
rabinovich fabrikant	yes	0.198	0.242	1.22	1.00
lorenz	yes	0.906	1.09	1.20	1.00
rossler	yes	0.0714	0.0763	1.07	0.99
burke shaw	yes	2.25	2.35	1.04	1.03
chen	yes	2.03	2.05	1.01	1.00

The published exponents disagree by a median of 41%; for thomas the dysts value is 21.7 times Sprott's. Under one method, every system whose control parameters match in both catalogues agrees to within 3%. The rest differ because the parameters differ — Sprott's Thomas uses $b = 0.18$ against a smaller dysts value, his Hadley $a = 0.25$, $F = 8$ against dysts' $a = 0.2$, $F = 9$ — which is why the corpus records both parameterisations rather than choosing one.

The method. `python/lyap_spectrum.py` implements Sprott's own procedure from chapter 5 of Sprott (2003): a fiducial trajectory advanced alongside k perturbed copies, modified Gram–Schmidt orthonormalisation of the separation vectors at each renormalisation, and the log of each vector's growth accumulated into the corresponding exponent. No Jacobian is needed, so one engine covers Sprott's 62 hand-written vector fields and the dysts catalogue alike. The 62 Sprott systems were ported to Python for the purpose and read line by line against the printed appendix. Every entry records the initial condition, control parameters, integration step and horizon it was computed from.

How an entry is judged. Three checks travel with every spectrum, none of which consults a published number: the sum of the exponents against the time-averaged divergence of the vector field, read straight off f by central differences; the exponent that theory requires to be exactly zero for an autonomous flow; and the standard error across independent ensemble members. The trace identity turned out to do double duty — integration error breaks it as surely as a faulty Gram–Schmidt does, which is how the integration step was calibrated per system rather than assumed. The corpus holds 181 systems (58 Sprott, 123 dysts), of which 137 pass all three checks; the rest are named with their reasons by `python/audit_corpus.py`. 6 dysts systems whose vector fields could not be vectorised without guessing at their calling convention are left out rather than risked: a wrong vector field produces a perfectly plausible exponent, which is the

failure this corpus exists to remove. A further 7 entries are kept but marked not chaotic at their recorded initial condition — the exponent is essentially zero there, so a ratio against it is noise — and excluded from the summaries below along with the IC-dependent ones.

Against Sprott's appendix. The recomputed values reproduce the printed appendix to a median absolute error of 8.9e-05: 35 of 58 usable systems within 1.5×10^{-4} of the published value, 50 within 2×10^{-3}, and every closed-form case at machine precision. That agreement is what licenses the corpus as a reference. Three systems resist it, and each was resolved differently:

- *ACT* (A.5.7) was a transcription error in the MATLAB catalogue: dz/dt takes $-\delta\alpha z$, and the α had been dropped, giving $-1.5z$ in place of $-2.7z$. The wrong system passed every internal check, because it is a perfectly good dynamical system; only the published comparison caught it, at 31% error. Corrected, it reads 0.162 against the published 0.163, with the third exponent −9.21 against -9.2060.
- *Rucklidge* (A.5.15) reads 0.192 against the published 0.0643, a factor of 3.0. The transcription matches the appendix and matches dysts' independent implementation; 25 different initial conditions all give 0.19; and recomputing from dysts' own parameters gives 0.192. The published value does not reproduce.
- *Moore–Spiegel* (A.5.11) reads 0.0933 against 0.112. Transcription, parameters and initial condition all match the appendix. Sixteen initial conditions spanning the basin, each with a 32-member ensemble, all land within 2% of 0.093; there is no second attractor to account for the difference.

Conservative systems carry a warning rather than a number. A dissipative system draws every trajectory onto one attractor, so its exponent belongs to the system. A conservative one does not: its state space is a patchwork of chaotic regions and regular islands that a trajectory never crosses between, so the exponent belongs to the starting point. Hénon–Heiles gave 0.0042 and 0.0449 in this work under nothing more than a different integration step, because the step moved the trajectory into a different region. Both are correct measurements of different things. The 22 such entries — Sprott's conservative maps and flows, and dysts' Hamiltonian systems — are flagged `icDependent` in the corpus and excluded from the summaries below. Arnold's cat map is deliberately not among them: it is conservative but uniformly hyperbolic, and the recomputation reproduces its closed form to 3×10^{-9}.

What it does to the results. Because the estimates do not depend on the reference, every ratio in this report can be re-divided by the recomputed reference without re-running the battery (`python/rescore_ratios.py`). Table 8 does that for the dissipative systems the corpus covers.

Table 8: Median estimate ÷ reference across dissipative systems, with the published reference and with the recomputed one. The estimates are identical in both columns; only the denominator changes.

Catalogue	Estimator	n	Median, published	Median, recomputed	Within 2×, published	Within 2×, recomputed
Sprott	Wolf	48	0.92	0.92	83%	85%
Sprott	Rosenstein	48	0.83	0.82	85%	88%
dysts	Wolf	109	0.98	1.08	74%	77%
dysts	Rosenstein	109	0.86	0.97	65%	73%

The Sprott results do not move: the recomputed references agree with the appendix, so every Sprott ratio in this report stands. The dysts results do. The published dysts spectra are 19% high at the median across the 117 systems in common, with only 28 within 10% of the recomputed value, and dividing by the corrected references changes what the estimators appear to be doing on that catalogue. Rosenstein's dysts median rises from 0.86 to 0.97, so most of the low bias reported for it on dysts was the reference; Wolf's rises from 0.98 to 1.08, so its apparent accuracy there was partly the same error in the other direction. On Sprott, Rosenstein's low bias survives the correction unchanged, at 0.82.

The corpus is `tests/reports/reference_spectra.json`. `quarctest.characterize` and `quarctest.characterize_series` take `Reference="recomputed"` to score against it directly, and record the choice in

every row's `referenceSource` and `referenceStatus`; the default remains `"published"`, so no committed table changes without saying so.

7.7 Sensitivity to observational noise

Design. Gaussian noise was added to each stored dysts series at 0, 0.5, 1, 2, 5 and 10% of that series' own standard deviation. `lyap_wolf`, `lyap_ros` and `corr_dim` were then re-run. This is measurement noise on the observable: the trajectory is unchanged, only the view of it is blurred. Every noisy series was analysed twice:

- **fixed:** delay and dimension taken from the clean series and held. What the estimator alone does with a noisier signal.
- **reembed:** delay and dimension re-estimated on the noisy series, as a practitioner would. What a real analysis produces.

Both arms see the same noise draw on the same realization, so any difference between them is the embedding and nothing else. The sweep covered 126 systems at R = 25, about 7 h on 32 workers.

Validation. At the 0% fixed cell, every value must equal what the battery produces for that realization. Across four systems at R = 25, all 700 compared values matched exactly.

Table 9: Median across systems of estimate ÷ reference, by noise level (% of series SD) and embedding arm.

Metric	Arm	0%	0.5%	1%	2%	5%	10%
Wolf	fixed	0.93	0.91	0.91	0.90	1.06	1.65
Wolf	reembed	0.93	0.93	0.88	0.86	0.86	1.40
Rosenstein	fixed	0.83	0.80	0.75	0.71	0.65	0.67
Rosenstein	reembed	0.83	0.79	0.74	0.68	0.58	0.50
Corr. dimension	fixed	1.14	1.17	1.21	1.31	1.61	1.92
Corr. dimension	reembed	1.14	1.16	1.22	1.33	1.72	2.25

Table 10: Share of systems whose ensemble-median estimate is within a factor of 2 of the reference, by noise level.

Metric	Arm	0%	0.5%	1%	2%	5%	10%
Wolf	fixed	69%	67%	66%	67%	67%	47%
Wolf	reembed	69%	68%	68%	67%	69%	60%
Rosenstein	fixed	60%	63%	59%	62%	58%	49%
Rosenstein	reembed	60%	64%	65%	65%	62%	42%

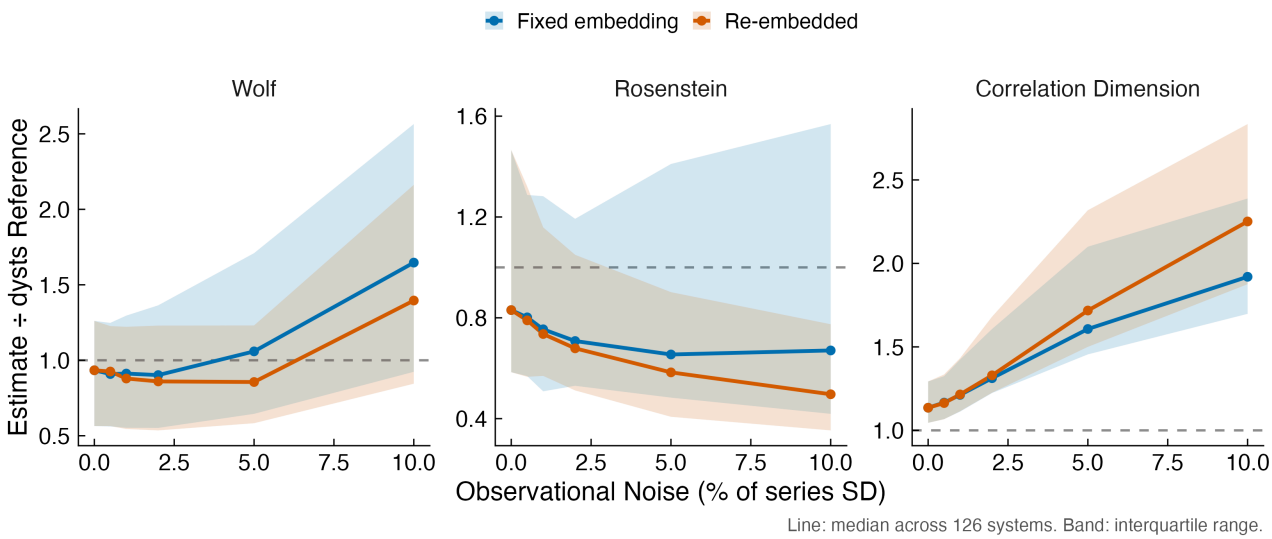


Figure 5: Noise sweep across all dysts systems. Line: median across systems. Band: interquartile range across systems. Blue holds the clean embedding; orange re-estimates it on the noisy series.

Four results stand out.

Below about 2% noise, nothing much happens. The two arms are indistinguishable and every metric stays close to its clean value. At these levels the protocol is robust and the embedding question does not arise.

Above 2%, the estimators fail in opposite directions. With the embedding held fixed, Wolf dips slightly and then overshoots, reaching 1.65 at 10%: noise inflates the measured stretch of neighbouring pairs. Rosenstein falls to 0.65 by 5% and levels off (0.67 at 10%), as noise lifts the floor of the divergence curve and flattens its initial slope. Two estimators of the same quantity degrading in opposite directions is worth knowing before choosing between them on noisy data. Averaging them would hide the problem rather than solve it.

Re-estimating the embedding shifts the Lyapunov estimates down, which helps or hurts depending on the estimator. Re-embedded, both exponents come out below their fixed-embedding values at every level from 1% upward. For Rosenstein that deepens an existing low bias: 0.50 against 0.67 at 10%. For Wolf at 5% it moves a near-correct median (1.06) below the reference (0.86). At 10% it partly cancels the overshoot (1.40 against 1.65): two errors offsetting, not an improvement. The correlation dimension moves the other way, inflating further when re-embedded (2.25 against 1.92).

By share of systems within 2×, the arms are closer than the medians suggest. With the embedding fixed, estimates spread more widely upward — the blue bands in Figure 5 — so up to 5% re-embedding keeps a similar share of systems within a factor of 2 for Wolf, and a slightly larger one for Rosenstein. At 10% the two estimators part: re-embedded Wolf keeps 60% of systems within 2× against 47% fixed, while re-embedded Rosenstein keeps 42% against 49%.

Re-embedding on noisy data is therefore not a remedy. It trades the fixed arm's wider spread for a downward shift in the median, and whether that shift helps depends on which way the estimator was already wrong. Noise and embedding dimension interact in reconstruction generally (Casdagli et al., 1991), and this is one concrete consequence.

Table 11: Embedding parameters chosen on the noisy series (reembed arm): median across systems.

Parameter	0%	0.5%	1%	2%	5%	10%
AMI delay (samples)	9.0	9.0	9.0	9.0	10.0	10.0
FNN dimension	5.0	5.0	5.0	6.0	6.0	7.0
Delay used	9.0	9.0	9.0	9.0	10.0	10.0
Dimension used	5.0	5.0	5.0	6.0	6.0	7.0

The embedding parameters were measured as outcomes rather than assumed. The AMI delay barely moves: it increased on 33% of systems between 0% and 10% noise. The FNN dimension climbed on 79% of systems. The re-embedding effect is a dimension effect, and the delay is not what changes. The design's opening premise was that noise inflates *both* parameters. The data corrected that.

7.8 The RQA family under the same noise

The sweep above left the recurrence-quantification metrics out, pending a fix to how diagonal lines were counted (`line_hist` counted a line once per point rather than once per line). With that fix in, the same design was run over the RQA family: the same 126 systems, the same R = 25, the same six levels and the same two arms, so that the two families are compared on identical noise draws. The RQA battery is far more expensive than the Lyapunov one — about 100 s per 16 000-sample series, dominated by the recurrence matrix — and the pass took 58 h on 32 workers, run as 21 restartable chunks of six systems and reassembled by `python/merge_noise_sweep_chunks.py`, which refuses to write unless every manifest system is present exactly once.

No reference, so each system is its own control. Unlike λ_1, the RQA metrics have no published value to score against, and their scales differ by orders of magnitude across systems (determinism runs 60 to 100, maximal line length from tens to thousands of samples). The quantity reported is therefore each system's value at a noise level divided by **its own clean value** at the same realization: how far noise moves the measurement from what the same series gave without it. Every system starts at 1 by construction. 125 of the 126 systems contribute; the RQA battery returns nothing at all for PanXuZhou, whose radius never reaches the target recurrence rate on any realization — clean or noisy — so there is no clean value to divide by.

Validation. At the 0% cell, the sweep's median over 25 realizations reproduces the committed characterization median over 100 to within half a percent for every RQA metric. The agreement is at the median across systems; individual systems can sit well off it for the two max-line statistics, as a maximum over 25 draws must relative to one over 100, and for the radius, which is re-selected per series to hit the recurrence target.

Table 12: RQA metrics under observational noise: median across systems of value ÷ that system's own clean value, by noise level (% of series SD) and embedding arm.

Metric	Arm	0%	0.5%	1%	2%	5%	10%
Determinism	fixed	1.00	1.00	1.00	0.99	0.95	0.87
Determinism	reembed	1.00	1.00	1.00	0.99	0.97	0.91
Laminarity	fixed	1.00	1.00	1.00	1.00	0.98	0.94
Laminarity	reembed	1.00	1.00	1.00	1.00	0.99	0.97
Mean line	fixed	1.00	1.00	0.99	0.97	0.83	0.59
Mean line	reembed	1.00	1.00	1.00	1.00	0.94	0.72
Max line	fixed	1.00	1.00	1.00	1.00	0.94	0.53
Max line	reembed	1.00	1.00	1.00	1.00	1.00	0.97
Max line (Theiler)	fixed	1.00	1.00	1.00	1.00	0.98	0.77
Max line (Theiler)	reembed	1.00	1.00	1.00	1.00	1.00	0.97
Line entropy	fixed	1.00	1.00	1.00	0.99	0.93	0.77
Line entropy	reembed	1.00	1.00	1.00	1.00	0.95	0.82
Vertical entropy	fixed	1.00	1.00	1.00	1.00	1.00	0.99
Vertical entropy	reembed	1.00	1.00	1.00	1.01	1.01	1.01
Weighted entropy	fixed	1.00	1.00	1.00	1.00	1.00	0.99
Weighted entropy	reembed	1.00	1.00	1.00	1.00	1.00	0.99
Radius	fixed	1.00	1.00	1.00	1.00	1.02	1.06
Radius	reembed	1.00	1.00	1.04	1.19	1.26	1.49

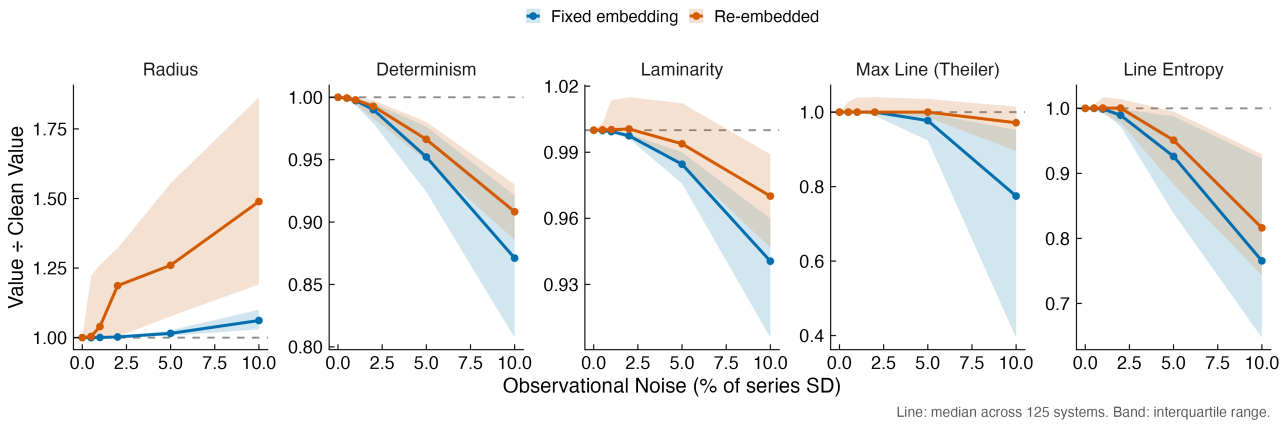


Figure 6: RQA metrics against observational noise, under the same two embedding arms. Line: median across systems of value ÷ clean value. Band: interquartile range. Blue holds the clean embedding; orange re-estimates it on the noisy series.

Three results stand out, and the third is the one that matters for this report.

The same 2% threshold. With the embedding fixed, no RQA metric moves more than about 1% from its clean value up to 2% noise. That is the same threshold the Lyapunov estimators showed, arrived at by a different route, which makes it a property of the data at this length and sampling rate rather than of either estimator family.

Past 2%, the family degrades gracefully and in a readable order. Nothing fails the way Wolf overshoots or Rosenstein loses its sign. At 10% noise and a fixed embedding, determinism is at 0.87 of clean and laminarity at 0.94, while the vertical-line metrics do not move at all (vertical entropy 0.99, weighted entropy 0.99). The diagonal-line-length statistics are the sensitive ones, and in the expected order: the raw maximum first (0.53), then the mean (0.59), then the entropy (0.77). The mechanism is visible in the ordering — noise breaks long diagonals into short ones before it changes how many points recur at all, so a statistic that depends on the longest line moves before one that counts lines, and one that counts lines moves before one that counts recurrent points.

Re-embedding helps RQA, and this is where the two families part. Re-estimating delay and dimension on the noisy series, noise inflates the FNN dimension on 80% of systems, and the radius has to grow to hold the 2.5% recurrence target — 1.49× its clean value at 10%. That larger radius re-joins the diagonals noise had broken. The raw maximum line recovers from 0.53 of clean to 0.97, and the Theiler-windowed one from 0.77 to 0.97; determinism recovers from 0.87 to 0.91. For Wolf and Rosenstein, Section 7.7 found the same re-embedding step *deepened* the error.

The practical consequence is sharp, because it is one decision with opposite signs. A practitioner who re-estimates the embedding on a noisy series — the ordinary, defensible thing to do — is protecting the RQA measurements and damaging the Lyapunov ones in the same step. An analysis that reports both families off one embedding pass cannot be right about both. Whether the embedding was chosen on the clean or the noisy signal therefore belongs in the methods section of any study that reports RQA and λ_1 together, and the two families may warrant separate embedding decisions.

8 Discussion

Which estimator to use. Neither is uniformly better. Wolf is closer to the reference on more dysts systems and has the smaller median error there. But its worst misses are orders of magnitude larger than Rosenstein's — roughly 89,944-fold against 362-fold on dysts — and it has a consequential tuning parameter. Rosenstein carries a consistent low bias of about 16% that its fit diagnostics help explain. Where a single catastrophic miss matters more than a small systematic bias, Rosenstein is the safer default. Where the dynamics are a map, Wolf should be run with `evolve` = 1.

Report the fit, not only the exponent. A Rosenstein exponent is a slope over a chosen window, and the window can move the answer by more than a factor of 2. Any analysis should report the fitted region and its curvature alongside λ_1.

Conservative, driven and weakly chaotic systems are hard for both. Small exponents sit near the estimation noise floor. A near-zero or wrong-sign estimate on such a system is not evidence against chaos.

Noise. Both estimators are usable up to about 2% observational noise relative to signal SD. Beyond that, the direction of the error depends on the estimator, and re-estimating the embedding on the noisy data shifts estimates rather than correcting them. For noisy data, embedding choices made on a smoothed or denoised copy, or methods designed for robustness to noise (Kantz, 1994), deserve a direct test.

Do not re-embed a noisy series for one family and report both. The RQA metrics share the Lyapunov estimators' 2% threshold (Section 7.8) and degrade gracefully past it, but they respond to re-embedding in the opposite direction: the inflated radius that a re-estimated embedding brings with it repairs the diagonals noise had broken, recovering the maximal line length from 0.53 of its clean value to 0.97, while the same step deepens the error in Wolf and Rosenstein. One embedding decision, opposite signs. A study reporting RQA and λ_1 from a single embedding pass on noisy data is getting one of the two wrong, and which one depends on a choice that usually goes unstated.

The reference matters as much as the estimator. The single largest change to the dysts results came from choosing the right reference value. Any benchmark of this kind should state exactly which published number it treats as ground truth and why.

9 Limitations and open issues

- **Reference uncertainty.** The dysts spectra are numerical estimates themselves, and Sprott's values are published to limited precision. Ratios bound estimator-plus-reference disagreement. Section 7.6 recomputes both catalogues under one method and quantifies the gap — the published dysts spectra run 19% high at the median — but the tables in this report still use the published values, and 6 dysts systems are not yet in the recomputed corpus. Two Sprott entries

(Rucklidge, Moore–Spiegel) reproduce neither from the appendix's own initial condition nor from any other, and stand in the corpus at the recomputed value with the disagreement recorded.

- **IC-dependent references.** Conservative systems have no single exponent to score against; 22 entries are flagged and excluded from the recomputed summaries. The flag is read off the computed spectrum — a divergence and exponent sum both within 10^{-3} of zero — with dysts' `hamiltonian` field as a supplement only, since that field misses volume-preserving systems such as Hénon–Heiles and SprottA. A separate flag marks 7 entries whose recorded initial condition is not chaotic at all (KawczynskiStrizhak, ForcedVanDerPol among them); their reference is essentially zero and they are excluded likewise. ForcedVanDerPol is the system on which Rosenstein's worst miss in the Summary was scored, which says something about that miss.
- **One observable, one sampling rate.** Each system is observed through a single scalar component at about 40 samples per period. Sensitivity to the choice of observable and to sampling rate was not tested here.
- **One embedding protocol.** Results are conditional on AMI/FNN for flows and the fixed rule for maps.
- **Heuristic scaling-region selection.** The Rosenstein region selector is a simple, uniform default, not a solved problem. The 200-sample cap binds often on dysts.
- **Conditioned ensembles.** A few systems (the damped and driven pendulums, labyrinth, Sakarya, HyperLu) lost realizations, and their survivors are conditioned on non-degeneracy.
- **Noise sweep scope.** Additive white Gaussian observational noise only, at R = 25, for both the Lyapunov family (Section 7.7) and the RQA family (Section 7.8). Dynamical noise, coloured noise, quantization and missing samples are untested.
- **RQA without a reference.** The RQA results are self-referential by necessity: with no published values to score against, each system is compared to its own clean measurement. The sweep can therefore say how far noise moves an RQA measurement, but not whether the clean measurement was right to begin with. The 1 system the RQA battery cannot characterize at all (PanXuZhou) is excluded rather than imputed.
- **Unexplained per-realization file.** The 13 September per-realization table in Dropbox does not reproduce current values for about a quarter of perturbed realizations, and its provenance is open.
- **Reproducibility of draws.** The single-channel dysts ensembles, except Sakarya, were drawn before the `crc32` seeding fix, so they cannot be regenerated bit for bit. The stored series are the record.

10 Data assets and how they can be reused

The corpora were built to be written once and read many times. Integration costs 2–4× what the whole metric battery costs per realization, so regenerating data per experiment would dominate every study.

Asset	Where	What it holds	Tracked in git
Sprott catalogue	`tests/matlab/+quarctest/sprott_catalog.m`	Equations, parameters, x_0, published λ_1; series generated on demand	Yes
dysts ensembles	`tests/reports/dysts/*.bin`; Dropbox `quarc-data/dysts`	126 systems × 100 × 16000 float64 samples (~1.6 GB)	No (manifest yes)
dysts manifest	`tests/reports/dysts/manifest.json`	Protocol, sampling rates, tangent-space and divergence exponents, spectra	Yes
Characterization reports	`characterization_full` and `characterization_dysts` (`.csv`, `.md`) in `tests/reports/`	Per-system, per-metric ensemble summaries	Yes
Per-realization tables	Local (Sprott, ~129 MB); Dropbox `derived/` (dysts, 27 MB)	Every realization × metric	No
Rosenstein fit curves	`tests/reports/curves_lyapunov.csv`	Divergence curves and fitted regions for a Sprott panel	Yes

Asset	Where	What it holds	Tracked in git
Wolf `evolve` sweep	`tests/reports/evolve_sweep.csv`	33 systems × 9 intervals, 200 realizations	Yes
Noise sweep (Lyapunov)	`tests/reports/noise_sweep.csv` (per-realization table, 15 MB, untracked)	126 systems × 6 levels × 2 arms	Summary yes
Noise sweep (RQA)	`tests/reports/noise_sweep_rqa.csv`; per-chunk tables in `tests/reports/noise_sweep_rqa/`	126 systems × 6 levels × 2 arms, nine RQA metrics	Yes

Because the series are stored, many further experiments cost only analysis time:

- **Series length:** truncate to any $N \leq 16000$. This is a column slice, and the within-realization comparison is exact.
- **Sampling rate:** decimate stored series to test coarser sampling (upsampling needs regeneration).
- **Alternative estimators** — for example Kantz's method (Kantz, 1994) or neural estimators — on literally identical data, for paired comparisons.
- **Embedding studies:** delay and dimension choices, Theiler windows, or denoised embeddings, evaluated against the same references.
- **Other noise models:** coloured noise, quantization, outliers, or missing samples, using the noise-sweep driver as a template.
- **Uncertainty quantification:** the per-realization tables give full distributions, not just medians, for every system and metric.

11 Reproducing this report

Every number in this document is computed at render time from the committed tables and manifest, so the report cannot drift from the data. The before/after reference comparison additionally reads the earlier report from git.

```
# Characterization (MATLAB)
matlab -batch "addpath('tests/matlab'); \
  [S,p] = quarctest.characterize(R=1000, N=4000); \
  quarctest.write_characterization(S, p, Tag='full')"
matlab -batch "addpath('tests/matlab'); \
  [S,p] = quarctest.characterize_series(R=100); \
  quarctest.write_characterization(S, p, Tag='dysts')"

# Sweeps (MATLAB); write the returned tables to tests/reports/
T = quarctest.evolve_sweep();
[S, per] = quarctest.noise_sweep(R=25);                    % Family="lyapunov" (default)
[S, per] = quarctest.noise_sweep(R=25, Family="rqa");      % run in chunks; see tests/README.md
python python/merge_noise_sweep_chunks.py                  % assemble the RQA chunks

# Figures and this report
Rscript R/make_figures.R
quarto render docs/lab-report/lyapunov_estimators_lab_report.qmd
```

Compiled on 2026-10-12 from the QUARC Nonlinear repository record: commits, committed tables, figures and the dysts manifest. Drafted with assistance from Claude (Anthropic).

12 Appendix A: per-system results, Sprott catalogue

Ensemble-median estimate ÷ published λ_1, with Rosenstein's fit R^2 and curvature.

System	Category	Wolf	Rosenstein	Fit R^2	Curvature
cubic_map	Noninvertible map	0.636	0.898	0.989	−0.34
cusp	Noninvertible map	0.77	0.842	0.983	−0.48
gauss_map	Noninvertible map	0.237	0.557	0.959	–
lcg	Noninvertible map	0.0405	0.0219	0.617	0.06
logistic	Noninvertible map	0.809	0.88	0.982	−0.45
pinchers	Noninvertible map	0.813	0.875	0.986	−0.42
ricker	Noninvertible map	0.884	0.848	0.983	−0.49
sine_circle	Noninvertible map	0.914	0.903	0.983	−0.47
sine_map	Noninvertible map	0.807	0.885	0.984	−0.43
burgers	Dissipative map	1.06	0.82	0.98	−0.48
delayed_logistic	Dissipative map	0.956	0.822	0.981	−0.53
dissipative_standard	Dissipative map	0.297	0.712	0.97	–
henon	Dissipative map	0.836	0.848	0.983	−0.47
holmes_cubic	Dissipative map	0.681	0.814	0.983	−0.47
ikeda	Dissipative map	0.628	0.791	0.984	−0.45
lozi	Dissipative map	0.765	0.817	0.984	−0.46
predator_prey	Dissipative map	0.998	0.844	0.982	−0.50
sinai	Dissipative map	0.387	0.879	0.982	−0.44
tinkerbell	Dissipative map	0.912	0.821	0.982	−0.52
arnold_cat	Conservative map	0.369	0.979	0.995	−0.50
chaotic_web	Conservative map	0.000735	0.0122	0.981	−0.29
chirikov	Conservative map	1.88	1.48	0.98	−0.50
gingerbreadman	Conservative map	0.764	0.769	0.98	−0.54
henon_area	Conservative map	0.279	3.3	0.98	−0.49
lorenz3d_map	Conservative map	1.1	0.957	0.981	−0.54
act	Autonomous flow	1.44	0.984	0.968	−0.66
burke_shaw	Autonomous flow	1.03	0.76	0.98	−0.56
chen	Autonomous flow	1.14	0.816	0.978	−0.56
chua	Autonomous flow	1.09	0.956	0.982	−0.51
complex_butterfly	Autonomous flow	0.796	0.661	0.98	−0.56
diffusionless_lorenz	Autonomous flow	1.02	0.753	0.98	−0.55
double_scroll	Autonomous flow	1.48	1.12	0.981	−0.52

System	Category	Wolf	Rosenstein	Fit R^2	Curvature
hadley	Autonomous flow	0.889	0.775	0.979	−0.56
halvorsen	Autonomous flow	1.08	0.898	0.978	−0.57
lorenz	Autonomous flow	1.41	0.931	0.979	−0.49
moore_spiegel	Autonomous flow	2.56	6.03	0.981	−0.54
rabinovich_fabrikant	Autonomous flow	0.924	0.847	0.981	−0.52
rigid_body	Autonomous flow	1.08	0.797	0.979	−0.55
rossler	Autonomous flow	0.941	0.82	0.997	−0.08
rucklidge	Autonomous flow	3.82	2.42	0.976	−0.56
simplest_cubic	Autonomous flow	1.44	4.15	0.981	−0.51
simplest_piecewise	Autonomous flow	1.16	1.53	0.983	−0.17
simplest_quadratic	Autonomous flow	0.93	0.904	0.996	−0.22
thomas	Autonomous flow	1.2	0.433	0.973	−0.59
windmi	Autonomous flow	0.903	0.636	0.976	−0.59
brusselator	Driven flow	0.962	0.675	0.969	−0.15
damped_pendulum	Driven flow	0.107	0.107	0.978	−0.48
driven_vdp	Driven flow	1.08	0.81	0.985	−0.42
duffing_two_well	Driven flow	0.517	0.138	0.853	−0.86
duffing_vdp	Driven flow	0.932	0.887	0.984	−0.50
rayleigh_duffing	Driven flow	0.915	0.885	0.98	−0.54
shaw_vdp	Driven flow	0.877	0.736	0.979	−0.54
ueda	Driven flow	1.04	0.681	0.981	−0.44
driven_pendulum	Conservative flow	0.000523	0.0561	0.975	−0.55
henon_heiles	Conservative flow	1.17	1.33	0.98	−0.54
labyrinth	Conservative flow	0.0308	0.0246	0.95	−0.67
nose_hoover	Conservative flow	3.17	5.65	0.983	−0.47
simplest_driven	Conservative flow	2.45	3.31	0.98	−0.55

13 Appendix B: per-system results, dysts catalogue

Ensemble-median estimate ÷ tangent-space reference, with Rosenstein's fit R^2 and curvature.

System	Category	Wolf	Rosenstein	Fit R^2	Curvature
Aizawa	Autonomous flow	1.04	0.864	0.987	−0.21
AnishchenkoAstakhov	Autonomous flow	1.21	1.52	0.997	−0.19
Arneodo	Autonomous flow	0.952	0.947	0.989	−0.41
AtmosphericRegime	Autonomous flow	0.993	0.578	0.99	−0.23

System	Category	Wolf	Rosenstein	Fit R^2	Curvature
BeerRNN	Autonomous flow	0.639	0.545	0.978	−0.53
BelousovZhabotinsky	Autonomous flow	0.286	0.17	0.973	−0.58
Blasius	Autonomous flow	1.21	1.18	0.996	−0.06
Bouali	Autonomous flow	1.23	2.42	0.964	−0.67
Bouali2	Autonomous flow	9.41	12.4	0.978	−0.56
BurkeShaw	Autonomous flow	0.991	0.765	0.98	−0.56
CaTwoPlus	Autonomous flow	0.662	0.717	0.998	−0.17
CaTwoPlusQuasiperiodic	Autonomous flow	0.151	6.81	0.983	−0.45
CellCycle	Autonomous flow	0.925	0.752	0.982	−0.51
CellularNeuralNetwork	Autonomous flow	1.2	1.58	0.98	−0.52
Chen	Autonomous flow	1.08	0.765	0.978	−0.57
ChenLee	Autonomous flow	1.61	3.87	0.979	−0.56
Chua	Autonomous flow	0.931	0.939	0.998	−0.14
CoevolvingPredatorPrey	Autonomous flow	3.1	0.851	0.981	0.05
Colpitts	Autonomous flow	0.867	0.839	0.996	−0.14
Coullet	Autonomous flow	0.912	0.833	0.985	−0.46
Dadras	Autonomous flow	2.08	2.78	0.972	−0.57
DequanLi	Autonomous flow	0.406	1.44	0.983	−0.50
ExcitableCell	Autonomous flow	0.165	0.221	0.985	−0.42
Finance	Autonomous flow	1.15	0.903	0.974	−0.60
GenesioTesi	Autonomous flow	0.979	0.883	0.998	0.02
GlycolyticOscillation	Autonomous flow	0.344	0.303	0.998	−0.05
GuckenheimerHolmes	Autonomous flow	1.44	2.96	0.98	−0.53
Hadley	Autonomous flow	0.655	0.667	0.995	0.15
Halvorsen	Autonomous flow	1.2	1	0.978	−0.58
HastingsPowell	Autonomous flow	1.16	0.678	0.985	−0.40
HenonHeiles	Autonomous flow	0.03	0.959	0.981	−0.51
HindmarshRose	Autonomous flow	1.22	1.31	0.993	−0.29
Hopfield	Autonomous flow	1.89	0.375	0.985	−0.24
HyperBao	Autonomous flow	2.25	2.78	0.976	−0.56
HyperCai	Autonomous flow	1.35	1.37	0.98	−0.55
HyperJha	Autonomous flow	2.9	1.08	0.989	−0.32
HyperLorenz	Autonomous flow	2.4	1.05	0.98	−0.51
HyperLu	Autonomous flow	1.66	1.71	0.979	−0.55

System	Category	Wolf	Rosenstein	Fit R^2	Curvature
HyperPang	Autonomous flow	1.26	1.36	0.98	−0.54
HyperQi	Autonomous flow	4.37	2.01	0.962	−0.67
HyperRossler	Autonomous flow	0.512	0.448	0.991	0.09
HyperWang	Autonomous flow	1.57	1.76	0.979	−0.55
HyperXu	Autonomous flow	1.41	0.692	0.975	−0.56
HyperYan	Autonomous flow	1.7	2.34	0.979	−0.52
HyperYangChen	Autonomous flow	2.39	3.09	0.977	−0.56
IsothermalChemical	Autonomous flow	0.0225	0.375	0.982	−0.44
ItikBanksTumor	Autonomous flow	1.19	0.762	0.972	−0.64
JerkCircuit	Autonomous flow	0.168	0.154	0.98	−0.56
KawczynskiStrizhak	Autonomous flow	0.168	1.77	0.999	−0.11
Laser	Autonomous flow	3.52	1.21	0.966	−0.38
LiuChen	Autonomous flow	2.19	0.717	0.978	−0.51
Lorenz	Autonomous flow	1.13	0.747	0.98	−0.50
Lorenz84	Autonomous flow	0.445	0.413	0.98	−0.55
Lorenz96	Autonomous flow	0.652	0.471	0.975	−0.59
LorenzBounded	Autonomous flow	0.885	0.788	0.989	−0.39
LorenzCoupled	Autonomous flow	0.819	0.703	0.98	−0.54
LorenzStenflo	Autonomous flow	0.983	0.739	0.98	−0.50
LuChen	Autonomous flow	0.522	0.402	0.972	−0.56
LuChenCheng	Autonomous flow	1.41	1.85	0.967	−0.62
MacArthur	Autonomous flow	0.0707	0.064	0.982	−0.53
MooreSpiegel	Autonomous flow	1.17	2.71	0.98	−0.53
MultiChua	Autonomous flow	1.09	1.22	0.996	−0.24
NewtonLiepnik	Autonomous flow	0.853	0.636	0.979	−0.56
NoseHoover	Autonomous flow	0.429	1.14	0.983	−0.49
PanXuZhou	Autonomous flow	–	−0.704	0.824	0.43
PehlivanWei	Autonomous flow	0.77	0.863	0.98	−0.48
Qi	Autonomous flow	1.52	1.1	0.977	−0.55
QiChen	Autonomous flow	1.11	0.849	0.979	−0.57
RabinovichFabrikant	Autonomous flow	0.882	1.03	0.992	−0.33
RayleighBenard	Autonomous flow	1.26	0.88	0.98	−0.51
RikitakeDynamo	Autonomous flow	0.911	0.836	0.998	−0.12
Rossler	Autonomous flow	0.897	0.782	0.998	−0.03

System	Category	Wolf	Rosenstein	Fit R^2	Curvature
Rucklidge	Autonomous flow	1.23	0.829	0.977	−0.58
Sakarya	Autonomous flow	0.846	0.63	0.98	−0.57
SaltonSea	Autonomous flow	0.953	0.962	0.989	−0.36
SanUmSrisuchinwong	Autonomous flow	0.91	0.724	0.98	−0.55
ShimizuMorioka	Autonomous flow	1.79	2.8	0.98	−0.55
SprottA	Autonomous flow	0.00402	7.43	0.994	−0.42
SprottB	Autonomous flow	0.981	0.735	0.98	−0.55
SprottC	Autonomous flow	0.981	0.752	0.98	−0.56
SprottD	Autonomous flow	0.853	0.728	0.998	0.15
SprottE	Autonomous flow	1.21	2.31	0.98	−0.53
SprottF	Autonomous flow	0.861	0.676	0.98	−0.55
SprottG	Autonomous flow	1.2	1.46	0.991	−0.32
SprottH	Autonomous flow	0.85	0.668	0.98	−0.56
SprottI	Autonomous flow	0.417	3.22	0.987	−0.31
SprottJ	Autonomous flow	1.57	1.29	0.995	0.07
SprottJerk	Autonomous flow	0.762	0.766	1	0.01
SprottK	Autonomous flow	0.968	1.05	0.991	−0.28
SprottL	Autonomous flow	1.89	9.58	0.979	−0.50
SprottM	Autonomous flow	1.14	2.45	0.98	−0.54
SprottN	Autonomous flow	1.17	0.936	0.995	0.05
SprottO	Autonomous flow	0.711	0.58	0.998	−0.02
SprottP	Autonomous flow	0.85	0.819	1	0.02
SprottQ	Autonomous flow	0.781	0.8	0.991	−0.34
SprottR	Autonomous flow	0.768	0.79	0.99	−0.26
SprottS	Autonomous flow	0.843	0.862	0.999	−0.10
SprottTorus	Autonomous flow	0.256	0.134	0.975	−0.56
Thomas	Autonomous flow	0.566	0.43	0.979	−0.56
ThomasLabyrinth	Autonomous flow	0.583	0.368	0.968	−0.60
Torus	Autonomous flow	–	–	0.0728	–
Tsucs2	Autonomous flow	0.0362	2.49	0.979	−0.56
VallisElNino	Autonomous flow	0.997	0.736	0.979	−0.48
WangSun	Autonomous flow	1.86	0.577	0.982	−0.37
WindmiReduced	Autonomous flow	0.522	0.583	0.996	−0.18
YuWang	Autonomous flow	0.601	0.355	0.978	−0.56

System	Category	Wolf	Rosenstein	Fit R^2	Curvature
YuWang2	Autonomous flow	0.575	0.561	0.996	0.15
ZhouChen	Autonomous flow	4.93	2.32	0.981	−0.46
CircadianRhythm	Driven flow	0.0273	2.1	0.981	−0.53
DoubleGyre	Driven flow	1.13	1.02	0.979	−0.55
Duffing	Driven flow	1.32	1.86	0.98	−0.53
FluidTrampoline	Driven flow	0.708	0.472	0.993	0.02
ForcedBrusselator	Driven flow	4.87	3.6	0.984	−0.10
ForcedFitzHughNagumo	Driven flow	0.392	4.43	0.969	−0.42
ForcedVanDerPol	Driven flow	−0.00039	0.00276	0.436	−0.06
OscillatingFlow	Driven flow	0.206	0.144	0.974	−0.59
TurchinHanski	Driven flow	2.91	2.19	0.993	−0.14
ArnoldWeb	Conservative flow	0.0814	0.0804	0.995	−0.12
BickleyJet	Conservative flow	0.277	22.3	0.981	−0.54
BlinkingRotlet	Conservative flow	0.149	0.0711	0.976	−0.52
BlinkingVortex	Conservative flow	0.191	0.139	0.974	−0.54
DoublePendulum	Conservative flow	1.11e-05	0.012	0.976	0.22
InteriorSquirmer	Conservative flow	0.268	0.167	0.977	−0.66
LidDrivenCavityFlow	Conservative flow	0.481	0.25	0.967	−0.64
NuclearQuadrupole	Conservative flow	0.817	0.335	0.979	−0.51
SwingingAtwood	Conservative flow	0.000212	0.0724	0.987	−0.18

Bibliography

Benettin, G., Galgani, L., Giorgilli, A., & Strelcyn, J.-M. (1980). Lyapunov characteristic exponents for smooth dynamical systems and for Hamiltonian systems; a method for computing all of them. Part 1: Theory. *Meccanica*, *15*(1), 9–20. https://doi.org/10.1007/BF02128236

Casdagli, M., Eubank, S., Farmer, J. D., & Gibson, J. (1991). State space reconstruction in the presence of noise. *Physica D: Nonlinear Phenomena*, *51*(1–3), 52–98. https://doi.org/10.1016/0167-2789(91)90222-U

Christiansen, F., & Rugh, H. H. (1997). Computing Lyapunov spectra with continuous Gram–Schmidt orthonormalization. *Nonlinearity*, *10*(5), 1063–1072. https://doi.org/10.1088/0951-7715/10/5/004

Eckmann, J.-P., & Ruelle, D. (1985). Ergodic theory of chaos and strange attractors. *Reviews of Modern Physics*, *57*(3), 617–656. https://doi.org/10.1103/RevModPhys.57.617

Fraser, A. M., & Swinney, H. L. (1986). Independent coordinates for strange attractors from mutual information. *Physical Review a*, *33*(2), 1134–1140. https://doi.org/10.1103/PhysRevA.33.1134

Gilpin, W. (2021). Chaos as an interpretable benchmark for forecasting and data-driven modelling. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, *1*.

Hairer, E., & Wanner, G. (1996). *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems: 14* (2nd ed., Vol. 14). Springer. https://doi.org/10.1007/978-3-642-05221-7

Kantz, H. (1994). A robust method to estimate the maximal Lyapunov exponent of a time series. *Physics Letters a*, *185*(1), 77–87. https://doi.org/10.1016/0375-9601(94)90991-1

Kennel, M. B., Brown, R., & Abarbanel, H. D. I. (1992). Determining embedding dimension for phase-space reconstruction using a geometrical construction. *Physical Review a*, *45*(6), 3403–3411. https://doi.org/10.1103/PhysRevA.45.3403

Rosenstein, M. T., Collins, J. J., & De Luca, C. J. (1993). A practical method for calculating largest Lyapunov exponents from small data sets. *Physica D: Nonlinear Phenomena*, *65*(1–2), 117–134. https://doi.org/10.1016/0167-2789(93)90009-P

Sprott, J. C. (2003). *Chaos and Time-Series Analysis*. Oxford University Press.

Takens, F. (1981). Detecting strange attractors in turbulence. In D. Rand & L.-S. Young (Eds.), *Dynamical Systems and Turbulence, Warwick 1980: Vol. 898. Dynamical Systems and Turbulence, Warwick 1980* (pp. 366–381). Springer. https://doi.org/10.1007/BFb0091924

Theiler, J. (1986). Spurious dimension from correlation algorithms applied to limited time-series data. *Physical Review a*, *34*(3), 2427–2432. https://doi.org/10.1103/PhysRevA.34.2427

Virtanen, P., Gommers, R., Oliphant, T. E., & others. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

Wald, A., & Wolfowitz, J. (1940). On a test whether two samples are from the same population. *The Annals of Mathematical Statistics*, *11*(2), 147–162. https://doi.org/10.1214/aoms/1177731909

Wolf, A., Swift, J. B., Swinney, H. L., & Vastano, J. A. (1985). Determining Lyapunov exponents from a time series. *Physica D: Nonlinear Phenomena*, *16*(3), 285–317. https://doi.org/10.1016/0167-2789(85)90011-9